CoAP test: laptop ↔ NodeMCU

The laptop asks the NodeMCU for its stats (`GET /stats`) and can switch the NodeMCU's blue LED on and off (`PUT /led`). No broker.

Files in this folder

`nodemcu_coap_server/nodemcu_coap_server.ino`	CoAP **server**. Arduino sketch (C++) for the NodeMCU.
`coap_client.py`	CoAP **client**. Python script for the laptop.

How it fits together

`coap_client.py (laptop)` $\xrightarrow{\text{GET /stats, PUT /led}}$ `NodeMCU (server)`

`coap_client.py (laptop)` $\xleftarrow{\text{2.05 Content + data}}$ `NodeMCU (server)`

	MQTT (previous test)	**CoAP** (this test)
Broker	Yes (Mosquitto)	**No**: laptop talks to the board directly
Who starts	NodeMCU **pushes** every 2 s	Laptop **asks**, NodeMCU **answers**
Transport	TCP, port 1883	**UDP**, port **5683**
Feels like	a group chat	a tiny website for devices (GET / PUT, like HTTP)
NodeMCU role	client (publisher)	**server**

The NodeMCU offers three things ("resources"):

- `GET /stats` → JSON: `uptime_s`, `rssi_dbm`, `free_heap`, `a0` (same meaning as in the MQTT test).
- `GET /led` → `on` or `off`.
- `PUT /led` with payload `on` / `off` → switches the blue LED.

`2.05 Content` in the answers is CoAP's version of HTTP's "200 OK".

Network: the laptop and the NodeMCU must be on the **same Wi-Fi**. No internet, no broker and **no proxy** are needed: the traffic is local. Tested working on the college Wi-Fi (IIITA). A phone hotspot also works.

Step 1: Install the libraries (once)

1. Arduino IDE → **Tools** → **Manage Libraries...** → search **CoAP simple library** → install the one by **Hirotaka Niisato**.
2. Laptop (PowerShell): `pip install aiocoap`

(On the college network, Arduino IDE needs its proxy set to download the library: **File** → **Preferences** → **Network** → **Manual proxy**, host `172.31.2.3`, port `8080`.)

Step 2: Open and edit the sketch

File → **Open** → `nodemcu_coap_server/nodemcu_coap_server.ino`, then set the Wi-Fi at the top:

```
const char* WIFI_SSID = "IIITA";   // Wi-Fi name
const char* WIFI_PASS = "";        // Wi-Fi password ("" if none)
```

Wi-Fi must be **2.4 GHz**. There is no broker or proxy setting: CoAP talks directly.

Step 3: Upload and get the NodeMCU's IP

1. Select board **NodeMCU 1.0 (ESP-12E Module)** and the COM port, then **Upload**.
2. **Tools** → **Serial Monitor**, **9600 baud**, press **RST**. It prints:

   ```
   Wi-Fi OK. CoAP server ready at  coap://172.29.0.146/stats
   >>> Use this IP in coap_client.py <<<
   ```

3. Note the IP. Tip: select it in the Serial Monitor and **Ctrl+C** to copy, instead of typing it.

Step 4: Ask for the stats from the laptop

```
cd C:\path\to\coap_test
python coap_client.py 172.29.0.146
```

Every 2 seconds:

```
13:17:11 2.05 Content  {"uptime_s":25,"rssi_dbm":-79,"free_heap":49624,"a0":4}
```

At the same time the Serial Monitor shows `GET /stats from <laptop IP> -> {...}`. Stop with **Ctrl+C**.

Step 5: Switch the LED from the laptop

```
python coap_client.py 172.29.0.146 led on
python coap_client.py 172.29.0.146 led off
```

Answer: `2.05 Content on` / `off`. The blue LED on the board follows, and the Serial Monitor shows `PUT /led 'on' from ....`

Troubleshooting

Symptom	Cause / fix
`no answer (timeout)`	Wrong IP (check Serial Monitor, press RST), board not powered, or the network blocks UDP between devices: use a phone hotspot for both.
Stuck on `Connecting to Wi-Fi.....`	Wrong name/password, 5 GHz network, or a login page network.
Gibberish in Serial Monitor	Baud must be 9600. One short line of gibberish right after RST is normal (boot message).
IP changed	The Wi-Fi gives out IPs automatically; it can change after a restart. Read it again from the Serial Monitor.

Result (tested)

Left: `coap_client.py` polling `/stats`, then `led on` and `led off`. Right: NodeMCU Serial Monitor showing each request it answered.

